

Using an LLM in a Formal Verification Loop to Discover Detailed-Balance-Satisfying Monte Carlo Algorithms: A Viability Assessment

May 2026

1 The Proposal

The system under consideration is DetailedBalanceChecker: a Mathematica symbolic verifier that formally proves or disproves whether a Monte Carlo algorithm satisfies detailed balance $T(i \rightarrow j) \pi(i) = T(j \rightarrow i) \pi(j)$. It enumerates all random-bit sequences via BFS, constructs the symbolic transition matrix, and tests the balance condition algebraically—giving a formal YES/NO proof, not a statistical test.

The proposal is to close a generative loop around this verifier: a large language model (LLM) writes a candidate algorithm as Mathematica code; the checker runs; the binary proof result and, on failure, the symbolic witness of imbalance, are fed back to the LLM; the LLM revises and retries. The goal is to discover new Monte Carlo algorithms for lattice particle/polymer systems that (a) satisfy detailed balance by formal proof and (b) exhibit dynamics qualitatively closer to real polymer physics than existing lattice algorithms such as Virtual Move Monte Carlo (VMMC) [1]. This report assesses where the proposal stands in the literature, identifies critical risks, and describes what a working prototype would look like.

2 Existing Algorithms and Correctness

Standard lattice algorithms. Kawasaki dynamics [2] (nearest-neighbour particle–hole swap with Metropolis acceptance) satisfies detailed balance trivially: the proposal is symmetric and the Metropolis criterion enforces balance. VMMC [1] grows clusters by recruiting neighbours with link probabilities $w_{fwd} = \max(1 - e^{\beta(e_{init} - e_{fwd})}, 0)$; a frustrated link aborts the move. This enforces *superdetailed balance*—a condition strictly stronger than detailed balance—and the checker verifies it analytically. VMMC approximates overdamped Brownian dynamics heuristically; its kinetics are physically plausible but not derived from the polymer equations of motion. Structurally related cluster algorithms include Swendsen–Wang [3] and the Wolff algorithm [4] for spin systems, and the Liu–Luijten geometric cluster algorithm [5] for off-lattice fluids.

Polymer Monte Carlo. The reptation family originates with de Gennes’ tube theory [6]. The slithering-snake (Kron) move removes a bond from one chain end and appends a new bond at the other end; it satisfies detailed balance by construction (uniform bond-vector distribution, symmetric forward and reverse moves) but is *non-ergodic* for self-avoiding walks on low-coordination lattices—a fact rigorously established by Madras & Sokal [7] for the pivot context and confirmed empirically for slithering-snake dynamics by Wittmer et al. [12]. This is the sharpest caution for the user’s project: the checker proves detailed balance but does not assess ergodicity, which is a separate necessary condition.

Configurational-Bias MC (CBMC; Siepmann & Frenkel [8]) grows a chain segment bead-by-bead with biased trial positions, correcting the bias via the ratio of Rosenbluth weights in the acceptance probability. It is the mathematically cleanest available template for satisfying detailed balance in chain-growth moves and is ergodic by construction. The pivot algorithm [7] applies a random lattice symmetry to the distal chain portion at a randomly chosen pivot; it satisfies detailed balance trivially and is provably ergodic on hypercubic lattices, but is inapplicable at dense packing. Houdayer’s wormhole move [9] constructs a non-local polymer-specific move from reptation steps that is provably detailed-balance satisfying and faster than CBMC by $O(N)$. End-bridging and

double-bridging MC [10, 11] cut and reconnect chains at interior segments; their acceptance criteria are non-trivial precisely because detailed balance must be preserved, a task the user’s checker could validate automatically.

It is also important to note that detailed balance is *sufficient* but not *necessary* for the Boltzmann stationary distribution. The weaker global balance condition is sufficient [13], and algorithms that exploit this—such as event-chain Monte Carlo [14], which satisfies only global balance via “lifting” variables and achieves rejection-free updates with faster dynamics—would *correctly fail* the user’s detailed-balance checker. This is a feature: it defines a precise search objective. It also means a strictly larger class of physically valid algorithms lies outside the user’s current search space, and those algorithms (ECMC variants) may be more efficient.

3 Prior Art: LLM-Driven Algorithm Discovery and Formal Verification

Table 1 summarises the most relevant published systems.

System	Year	Key feature	Physics MC?
FunSearch [15]	2023	LLM + evolutionary empirical evaluator	No
AlphaEvolve [16]	2025	Full codebase evolution, empirical test suites	No
AI Scientist [17]	2024	LLM full research loop; empirical ML benchmarks	No
SGA [18]	2024	LLM hypotheses + differentiable simulation	No
POISE [19]	2025	LLM discovers RL algorithms; consistency checks	No
Clover [20]	2024	LLM + Dafny SMT formal verification	No
LLMLOOP [21]	2025	LLM + iterative compile/test feedback	No
Norm. flows for LFT [22]	2021	Neural network MCMC proposals for lattice QFT	Partial

Table 1: Most relevant published systems. None applies LLM generation to physical MC algorithm design with a formal symbolic correctness oracle.

FunSearch [15] is the closest methodological ancestor: an LLM proposes Python functions; an automated evaluator scores them; the best are evolved. It discovered new cap-set solutions and improved bin-packing heuristics. AlphaEvolve [16] (2025) extends this to evolving entire codebases and found the first improvement to Strassen matrix multiplication in 56 years—but relies on empirical performance scoring, not formal physical correctness. Neither has been applied to Monte Carlo algorithm design.

In the formal-verification literature, Clover [20] and LLMLOOP [21] demonstrate that LLM+symbolic-checker loops are effective in software engineering contexts (Lean/Dafny proofs, static analysis). A salient finding: “compile-execute-revise loops are insufficient for scientific computing, where a program may run successfully without producing correct results” [21]. This directly motivates the user’s symbolic checker over a runtime test suite. However, none of these systems has been applied to physical simulation algorithm design.

The combination—LLM proposer, formal detailed-balance checker as oracle, binary proof fed back to guide revision, targeting polymer/soft-matter physics—does not appear in the published literature. The gap is real.

4 Critical Assessment

What is genuinely strong. The symbolic checker provides a qualitatively better feedback signal than empirical tests. A symbolic counterexample—the specific state pair (i, j) and the non-zero residual expression—is far more

informative to the LLM than a failed KL divergence test. The existing verified examples (Kawasaki, VMMC and variants) constitute ready-made few-shot examples that teach the LLM both the required code structure and the physical intuitions that have previously worked. The abstract-parameter capability of the checker (coupling and field functions as free symbols) means new algorithms are verified for all possible coupling constants simultaneously. The methodological novelty over FunSearch is real: replacing an empirical performance evaluator with a formal mathematical oracle is a qualitative upgrade in the correctness guarantee.

Ergodicity is not checked. This is the most serious risk. An algorithm that satisfies detailed balance but is non-ergodic (as the slithering-snake is) will pass the checker with a clean PASS while producing incorrect equilibrium sampling. The slithering-snake precedent is a direct warning: the checker would certify it. Any prototype must address ergodicity separately, either by restricting to move families that are ergodic by construction (CBMC-style full-chain regrowth) or by implementing an ergodicity check as a BFS over the transition graph.

The search space is vast and unstructured. Unconstrained LLM generation from a blank prompt will overwhelmingly produce syntactically valid but physically incoherent code. The correct approach is constrained search: give the LLM a skeleton algorithm—a physically motivated structure with a specific missing or modifiable component—and ask it to fill in that component. The VMMC codebase itself documents that human experts introduce subtle errors (expanded neighbour shell, double-test problem on $L = 2$, `Piecewise` vs `Max`) that the checker catches. LLM generation will fail in analogous and new ways, requiring many iterations. This is acceptable if the loop is fast; the checker’s runtime of $\sim 50\text{--}100\text{ s}$ for VMMC at `MaxBitString=1111111111` is short enough to iterate hundreds of times in a single session.

The physical target is underspecified. “Closer to base reality of polymer interaction” requires operationalisation. What observable? Mean-squared displacement scaling ($\langle r^2(t) \rangle \propto t^{1/2}$ for Rouse, $\propto t^{1/4}$ for reptation), stress relaxation modulus, or chain renewal time scaling? Without a measurable kinetic target, neither the LLM nor the investigator can determine whether a passing algorithm is an improvement over VMMC. Detailed balance is necessary for correct equilibrium sampling; it says nothing about kinetics.

Competing approaches are strong. CBMC already provides a principled mathematical template for any chain-growth move. AlphaEvolve could in principle be applied to this problem if given a suitable evaluator, with far more computational resources. The user should be prepared to articulate what physical move CBMC cannot express that the target algorithm would. Event-chain MC [14] achieves rejection-free updates with correct equilibrium statistics and dynamics often closer to Brownian motion; it lies outside the detailed-balance search space but may be the more powerful tool for the physical goals stated.

LLMs can contribute but are not magic. The value of the LLM in this loop is primarily as a rapid hypothesis generator: it can propose syntactically valid code variants quickly and incorporate the symbolic counterexample into its revision. It cannot reason reliably about whether a proposed move is ergodic, whether its kinetics will match any particular dynamical regime, or whether the acceptance criterion correctly cancels a sampling bias without being given the relevant mathematical formula.

5 Publication Viability and Prototype

The gap is real and the minimum viable publication consists of three things: a new algorithm discovered via the LLM-checker loop that (a) passes the formal symbolic checker, (b) is demonstrably ergodic, and (c) exhibits a kinetic observable not reproduced by VMMC or Kawasaki on the same lattice model. The methodological contribution—the loop itself—is novel and worth documenting, but it will not carry a paper alone without the algorithmic result. Suitable venues include *J. Chem. Phys.*, *J. Chem. Theory Comput.*, *Phys. Rev. E*, or a methods letter at the AI-physics interface (*Mach. Learn.: Sci. Technol.*).

A working prototype has four components: **(1)** A structured few-shot prompt containing two passing examples (Kawasaki, VMMC) in full, a physical description of the target move class, the most recent code attempt, and the checker’s symbolic counterexample if the attempt failed. **(2)** Checker execution in symbolic mode (`check.wls, Mode=Symbolic, MaxBitString=11111` or higher). **(3)** Failure-analysis extraction: parse the checker output to identify the specific (i, j) pair and residual expression, and include these verbatim in the next prompt. **(4)** Ergodicity verification: for each PASS result, run a BFS over the transition graph from all reachable states to confirm irreducibility.

The most productive immediate target is a CBMC-style lattice chain-regrowth move: delete a contiguous chain segment, regrow bead-by-bead with biased sampling, and correct the bias via Rosenbluth weights in the acceptance

criterion. The Rosenbluth weight formula can be given to the LLM explicitly as a design pattern; the checker then verifies that the LLM’s implementation actually achieves cancellation. This is ergodic by construction, physically motivated (Siepmann & Frenkel 1992 in the lattice context), and distinct from VMMC in its kinetic character.

6 An Extended Loop Architecture

The preceding sections identified several gaps in the minimal LLM–checker loop: ergodicity is not verified, there is no continuous signal for kinetic quality, and the LLM is asked to guess acceptance criteria that could instead be derived exactly. This section develops a concrete plan for a richer loop that addresses all three.

Ergodicity verification from the transition matrix

The checker already constructs the full symbolic transition matrix T_{ij} . Ergodicity of the Markov chain is equivalent to strong connectivity of the directed transition graph: every state must be reachable from every other state. The support matrix $S_{ij} = 1$ if $T_{ij} \neq 0$ (structurally nonzero, as established by `=== 0` in Mathematica’s symbolic algebra), and $S_{ij} = 0$ otherwise. A standard result from graph theory guarantees that the matrix power $(I + S)^{n-1}$ has a strictly positive (i, j) entry if and only if state j is reachable from state i in at most $n - 1$ steps [23]. The ergodicity check is therefore:

```
CheckErgodicity[T_List] := Module[{n, S, R},
  n = Length[T];
  (* Support: 1 if symbolically nonzero, 0 otherwise *)
  S = Map[If[# === 0, 0, 1] &, T, {2}];
  (* Reachability via matrix power (exact for integer matrices) *)
  R = MatrixPower[IdentityMatrix[n] + S, n - 1];
  If[Min[Flatten[R]] > 0,
    Print["ERGODIC: transition graph strongly connected."];
    True,
    (* Report the first unreachable pair *)
    With[{bad = First[Position[R, 0, {2}, 1]]},
      Print["NOT ERGODIC. State ", bad[[2]],
        " unreachable from state ", bad[[1]]];
      False]
  ]
```

For the state-space sizes the checker examines (typically $n \approx 100$ states for small lattices), $n - 1$ matrix multiplications of integer matrices complete in milliseconds. The check should be inserted immediately after a PASS from the detailed-balance checker; an algorithm that passes both is a valid candidate for further evaluation.

The check gives *generic* ergodicity: if T_{ij} is structurally nonzero but zero at a particular parameter value (e.g. $J = 0$), the check correctly identifies the transition as generically available. For degenerate parameter regimes, a separate numerical check at problematic parameter values is advisable, but generic ergodicity is the physically relevant condition for a non-trivial coupling constant.

Physical fidelity as a continuous optimisation signal

Detailed balance certifies that the stationary distribution is Boltzmann; ergodicity certifies that the chain reaches it. Neither condition says anything about whether the *path* taken resembles real physical dynamics. A third signal is therefore needed: a scalar score measuring how closely the MC transition matrix T^{MC} matches the physical transition matrix T^{phys} derived from the actual equations of motion of the modelled system.

Computing T^{phys} for the lattice model. For a lattice gas with pairwise interactions, the physical dynamics is overdamped Brownian motion projected onto lattice sites. The transition rates under this projection are given by Glauber dynamics [24]: for a single-particle hop from configuration A to B differing by one lattice step,

$$k_{A \rightarrow B}^{phys} = \nu_0 \cdot \frac{e^{-\beta \Delta E / 2}}{2 \cosh(\beta \Delta E / 2)},$$

which is the symmetric Glauber rate (here $\Delta E = E_B - E_A$). For cluster moves of n particles under Stokes drag, the physical rate scales as $k_{cluster}^{phys}(n) \propto D_n \propto 1/n$ in the appropriate dimension (with corrections for non-spherical clusters). For the small 2D lattice systems the checker examines, T^{phys} is a fully enumerable matrix: every pair

of states connected by a one-particle hop or a physically plausible multi-particle rearrangement has a calculable rate. Numerically this amounts to evaluating ΔE for each transition pair at specific parameter values (β, J) —a negligible computation for the $n100$ state spaces involved.

Comparing T^{MC} and T^{phys} . A direct comparison of matrix entries is confounded by the timescale ambiguity: the MC step is not the same as the physical time unit. The appropriate comparison is therefore of the *shapes* of the two matrices as row-conditional distributions. For each initial state i , define the normalised physical rates $\tilde{T}_{ij}^{phys} = k_{ij}^{phys} / \sum_j k_{ij}^{phys}$ and the MC proposal probabilities $\tilde{T}_{ij}^{MC} = T_{ij}^{MC} / \sum_j T_{ij}^{MC}$. The fidelity score is

$$\mathcal{F} = - \sum_i \pi_i \sum_j \tilde{T}_{ij}^{phys} \log \frac{\tilde{T}_{ij}^{phys}}{\tilde{T}_{ij}^{MC}},$$

the negative KL divergence averaged over states weighted by the equilibrium distribution. Higher (less negative) is better; $\mathcal{F} = 0$ means perfect shape match. The score is undefined if $T_{ij}^{MC} = 0$ for a transition with $\tilde{T}_{ij}^{phys} > 0$: the MC algorithm cannot execute a physically possible transition. This is a hard failure and should be treated as disqualifying.

An alternative metric that avoids the undefined-log problem is comparison of the non-trivial eigenvalue spectra: $|\lambda_k^{MC} - c \lambda_k^{phys}|$ for a fitted scaling constant c . This measures whether the relaxation timescales of the two dynamics have the same *ratios*, even if their absolute scales differ. Matching the ratio λ_2/λ_3 (two slowest non-trivial modes) is a stringent test of kinetic fidelity.

Feasibility and practical implementation. For a 2×2 lattice with 2 particle types, the full state space has ~ 20 – 50 states, and all computations above complete in under a second. For 3×3 lattices (~ 1000 states), numerical evaluation of T^{phys} at a few parameter sets ($\beta \in \{0.5, 1.0, 2.0\}$, $J \in \{-1, 0, +1\}$) takes minutes. This is the right regime: the physical fidelity signal should be evaluated numerically at a handful of representative parameter values—not symbolically—and averaged. The symbolic checker runs first (exact, parameter-free); the physical fidelity scorer runs only on algorithms that pass both the DB check and the ergodicity check.

The acceptance rate problem and rejection-free variants

A specific and important inefficiency in standard VMMC is the following. A cluster of n particles can be initiated from any of its n members, so it is effectively attempted n times per MC sweep. Under Stokes drag the physical diffusion coefficient of a compact cluster scales as $D_n \propto 1/n$ in 3D (and analogously in 2D), so the physical displacement rate is $\propto 1/n$. For the MC attempt rate ($\propto n$) times the acceptance probability to equal the physical rate ($\propto 1/n$), the acceptance probability must scale as $\propto 1/n^2$. This is a severe penalty: for clusters of 10 particles the typical acceptance rate is already $\sim 1\%$, making large-cluster moves essentially decorative.

Note that this is distinct from the frustrated-link rejection, which is a consequence of detailed balance and is in principle unavoidable given the move structure. The Stokes inefficiency is a consequence of the *proposal* distribution (uniform seed choice combined with fixed step size) not matching the physical rate distribution. It can in principle be corrected without modifying the acceptance criterion.

Proposal-level corrections. The natural fix is to *weight the seed selection inversely by cluster size*: assign seed particle p a selection probability proportional to $1/n_p$ where n_p is the size of the cluster that would form from p . This biases the algorithm toward proposing small-cluster moves with the correct relative frequency. The detailed-balance check must then be re-derived; the checker would verify whether any specific implementation of this idea maintains balance. This is precisely the kind of modification the LLM is well placed to propose: it is a small, local change to the seed-selection code with a clear physical motivation, and the symbolic checker immediately confirms or refutes whether balance is preserved.

Automatically deriving the acceptance criterion. A more radical approach exploits the symbolic checker directly. Observe that the checker already computes $q(i \rightarrow j)$ —the probability of the proposal kernel mapping state i to state j —as a symbolic expression. Given q , the unique Metropolis acceptance probability that satisfies detailed balance is

$$a(i \rightarrow j) = \min \left(1, \frac{q(j \rightarrow i)}{q(i \rightarrow j)} e^{-\beta \Delta E_{ij}} \right).$$

Mathematica can compute the ratio $q(j \rightarrow i)/q(i \rightarrow j)$ symbolically from the already-enumerated transition paths. This means the workflow can be restructured: the LLM proposes only the *proposal distribution* (the cluster growth

rule, without an acceptance step); Mathematica derives the acceptance criterion automatically and inserts it; the checker then verifies the combined algorithm. This separates the creative task (proposing a physically motivated cluster growth rule) from the mechanical task (computing the acceptance probability). The acceptance rate $\bar{A}(\beta, J)$ can then be computed symbolically for small systems:

$$\bar{A}(\beta, J) = \sum_{i,j} \pi_i(\beta, J) q(i \rightarrow j) \min\left(1, \frac{q(j \rightarrow i)}{q(i \rightarrow j)} e^{-\beta \Delta E_{ij}}\right)$$

and maximised over the free parameters in the growth rule. This turns algorithm discovery into an explicit optimisation problem with an exact objective function, computable in Mathematica for any candidate proposal the LLM generates.

The target is a growth rule for which $\bar{A}$ is high (approaching 1) and $\mathcal{F}$ (the physical fidelity score defined above) is also high. These two objectives can conflict: a proposal that always accepts may do so by making moves that are physically unmotivated, giving low $\mathcal{F}$. Managing this tension is where physical intuition (and the LLM’s knowledge of Brownian dynamics) becomes essential.

How an LLM should exploit Mathematica’s symbolic engine

The standard LLM–code loop (write code, run it, observe output, fix code) uses the execution environment as an oracle for binary correctness. The present system can go further, because Mathematica is a computer algebra system, not merely a runtime. The LLM can interact with it in several qualitatively richer ways.

Receiving algebraic error witnesses. When the detailed-balance checker fails, it does not merely say “wrong”; it returns the specific symbolic expression $T_{ij}\pi_i - T_{ji}\pi_j$ that is nonzero for a specific state pair. This expression is a polynomial in $e^{\beta J_{ab}}$ terms. The LLM can parse this expression and identify the structural cause of the imbalance. For example, if the residual is $e^{\beta J_{12}} - e^{\beta J_{13}}$, the algorithm is treating bonds of type (1,2) and (1,3) asymmetrically when it should treat them identically. This is a diagnostic far richer than any runtime error message.

Proposing and testing parametric families. Rather than proposing a single algorithm, the LLM can propose a *parametric family* with free parameters (e.g. the frustration threshold, a weighting exponent, a cutoff distance). Mathematica computes the acceptance rate and fidelity score as symbolic functions of these parameters, and the optimal values can be found by `Maximize` or `FindMaximum`. This allows the LLM to propose general algorithmic structures and let Mathematica tune them—a more productive division of labour than asking the LLM to guess optimal parameter values.

Symbolic consistency checks before running the full checker. For a candidate algorithm, certain necessary conditions for detailed balance can be checked symbolically in microseconds, without running the full BFS enumeration. For example: are the transition probabilities normalised (each row sums to 1)? Is $T_{ij} = 0$ whenever $E_j - E_i > E_{max}$ (energy monotonicity)? Is $T_{ij}/T_{ji} = e^{-\beta \Delta E_{ij}}$ for all directly connected pairs (local balance)? The LLM can be instructed to include these checks in its generated code as self-diagnostic assertions, catching structural errors before the expensive BFS enumeration runs.

Guided mutation of the acceptance criterion. Once the symbolic ratio $q(j \rightarrow i)/q(i \rightarrow j)$ is computed, the LLM can be prompted: “the current acceptance ratio is $R = [expression]$; propose a modification to the growth rule that makes R closer to 1 for large clusters while keeping the expression valid as a probability”. This is a well-posed algebraic question that the LLM can reason about in symbolic terms, guided by the exact expression from Mathematica rather than a vague description of the imbalance.

Connection to embedded LLM theory

The theoretical framework developed in the companion document `embedded_LLM.tex` [25, 26, 27] identifies three mechanisms by which embedding an LLM in an executable environment improves performance: unambiguous external feedback (the environment cannot be fooled by fluent text), offloading of formal computation (the interpreter does exactly what the LLM approximates poorly), and error localisation (failures are pinpointed rather than compounding).

The present system is a particularly rich instantiation of all three. The detailed-balance checker provides feedback the LLM cannot hallucinate: the algebraic expression $T_{ij}\pi_i - T_{ji}\pi_j$ is a theorem, not a probability. The acceptance rate $\bar{A}(\beta, J)$ and fidelity score $\mathcal{F}$ are formal computations that Mathematica evaluates exactly in seconds and the

LLM could not reliably compute internally. And the symbolic counterexample localises the error to a specific state pair and a specific algebraic term.

The proposed system adds a *fourth* mechanism not discussed in the embedded-LLM literature: **symbolic reasoning feedback**. A Python traceback tells the LLM “your code crashed at line 12”. The symbolic checker tells the LLM “the detailed balance condition fails for states (3, 7) because the expression $e^{\beta J_{12}} - e^{\beta J_{13}}$ is nonzero”. This is not a runtime error; it is a mathematical statement about the structure of the algorithm. The LLM can reason about it algebraically—recognising which part of the growth rule generated the J_{12} term and which generated the J_{13} term, and modifying the code to symmetrise them. This is qualitatively beyond what execution feedback in standard programming contexts can provide: the oracle speaks in the same algebraic language as the problem domain.

This feature also resolves a key limitation identified in the embedded-LLM literature: that “execution loops are insufficient for scientific computing, where a program may run correctly without producing correct results” [21]. The symbolic checker does not test outcomes; it tests the underlying mathematical relationship. A program that runs without error and produces numerically plausible-looking output but violates detailed balance will be identified as incorrect—exactly the failure mode that runtime tests cannot catch. The combination of three independent formal verifiers (detailed balance, ergodicity, physical fidelity) provides a layered guarantee that no existing LLM algorithm discovery system approaches.

7 Conclusion

The proposal is scientifically sound and fills a genuine gap. The core innovation—using a formal mathematical oracle in place of an empirical evaluator in an LLM discovery loop—is both conceptually sharp and practically feasible with the existing checker infrastructure. The principal risks are the unchecked ergodicity problem, the underspecification of the physical target, and the breadth of the search space. None is fatal; all require explicit mitigation. If the LLM loop produces one new algorithm that passes the formal checker, is ergodic, and distinguishes itself kinetically from VMMC, there is a credible publication.

References

- [1] Whitelam, S. & Geissler, P. L. (2007). Avoiding unphysical kinetic traps in Monte Carlo simulations of strongly attractive particles. *J. Chem. Phys.* **127**, 154101.
- [2] Kawasaki, K. (1966). Diffusion constants near the critical point for time-dependent Ising models. *Phys. Rev.* **145**, 224.
- [3] Swendsen, R. H. & Wang, J.-S. (1987). Nonuniversal critical dynamics in Monte Carlo simulations. *Phys. Rev. Lett.* **58**, 86.
- [4] Wolff, U. (1989). Collective Monte Carlo updating for spin systems. *Phys. Rev. Lett.* **62**, 361.
- [5] Liu, J. & Luijten, E. (2004). Rejection-free geometric cluster algorithm for complex fluids. *Phys. Rev. Lett.* **92**, 035504.
- [6] de Gennes, P. G. (1971). Reptation of a polymer chain in the presence of fixed obstacles. *J. Chem. Phys.* **55**, 572.
- [7] Madras, N. & Sokal, A. D. (1988). The pivot algorithm: a highly efficient Monte Carlo method for the self-avoiding walk. *J. Stat. Phys.* **50**, 109.
- [8] Siepmann, J. I. & Frenkel, D. (1992). Configurational bias Monte Carlo: a new sampling scheme for flexible chains. *Mol. Phys.* **75**, 59.
- [9] Houdayer, J. (2002). The wormhole move: a new algorithm for polymer simulations. *J. Chem. Phys.* **116**. arXiv:cond-mat/0111087.
- [10] Pant, P. V. K. & Theodorou, D. N. (1995). Variable connectivity method for the atomistic Monte Carlo simulation of polydisperse polymer melts. *Macromolecules* **28**, 7224.
- [11] Karayiannis, N. C., Mavrantzas, V. G. & Theodorou, D. N. (2002). A novel Monte Carlo scheme for the rapid equilibration of atomistic model polymer systems. *Phys. Rev. Lett.* **88**, 105503.

- [12] Wittmer, J. P. et al. (2003). Dynamical properties of the slithering-snake algorithm: a numerical test of the activated-reptation hypothesis. *Eur. Phys. J. E* **10**, 369. arXiv:cond-mat/0212433.
- [13] Manousiouthakis, V. I. & Deem, M. W. (1999). Strict detailed balance is unnecessary in Monte Carlo simulation. *Phys. Rev. E* **59**, 2.
- [14] Bernard, E. P., Krauth, W. & Wilson, D. B. (2009). Event-chain Monte Carlo algorithms for hard-sphere systems. *Phys. Rev. E* **80**, 056704.
- [15] Romera-Paredes, B. et al. (2023). Mathematical discoveries from program search with large language models. *Nature* **628**, 73.
- [16] Google DeepMind (2025). AlphaEvolve: a coding agent for scientific and algorithmic discovery. arXiv:2506.13131.
- [17] Lu, C. et al. (2024). The AI Scientist: towards fully automated open-ended scientific discovery. arXiv:2408.06292.
- [18] Ma, P. et al. (2024). LLM and simulation as bilevel optimizers: a new paradigm to advance physical scientific discovery. *ICML 2024*. arXiv:2405.09783.
- [19] From AI assistant to AI scientist: autonomous discovery of LLM-RL algorithms with LLM agents. (2025). arXiv:2603.23951.
- [20] Clover: closed-loop verifiable code generation. (2024, AAAI adjacent).
- [21] LLMLOOP: automating refinement of LLM-generated code through iterative feedback. *ICSME* (2025). arXiv:2603.23613.
- [22] Albergo, M. S. et al. (2021). Introduction to normalizing flows for lattice field theory. arXiv:2101.08176.
- [23] Horn, R. A. & Johnson, C. R. (1985). *Matrix Analysis*. Cambridge University Press.
- [24] Glauber, R. J. (1963). Time-dependent statistics of the Ising model. *J. Math. Phys.* **4**, 294.
- [25] Gao, L. et al. (2023). PAL: Program-aided language models. *Proc. ICML*, PMLR 202:10764.
- [26] Yao, S. et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*.
- [27] Shinn, N. et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS 2023*.